

迭代一中间产物-汇总

1. Ragas 功能描述

1.1 建立评估样本数据集

用户可以通过 `EvaluationDataset` 将来源于代码、JSON 文件等媒介的多个评估样本组织成样本数据集，便于评估模型使用。同时，评估样本数据集可以轻松导出多种持久化格式，包括 JSON、DataFrame 等格式，以便其他外部应用使用。

1.2 评估结果存储

通过结构化管理样本和自动关联评估得分，支持高效的分析、报告生成和模型优化过程。

1.3 评估模型对话样本数据集

根据给定的用户与模型交互的内容以及参考内容组成的评估样本的数据集，向用户返回数据集中样本在给定指标下的评估得分。

1.3.1 基于 LLM 评估模型

使用另一个 LLM 生成的回复作为参考基线，提供更准确、提供更接近人类对话水准的评估。

1.3.2 传统指标评估模型

通过 `BleuScore`、`Token cost` 等传统指标，独立于 LLM 进行评估，适用于多种应用场景，确保评估结果的多样性和可靠性，提供多维度的性能分析。

1.3.3 自定义指标评估模型

使用定义好的接口，轻松实现自定义指标评估，满足特定业务需求。

1.4 生成测试数据集

Ragas 支持多种样本数据集的生成方式，从自定义到 LLM 驱动自动数据集生成，以及下载平台的公开数据集。

1.4.1 自定义测试样本数据集

从自定义的用户模型交流样本构建数据集。通过自定义用户模型交流样本，构建特定领域的专用数据集。

1.4.2 自动测试数据生成

Ragas 能自动生成覆盖广泛场景的测试数据集，支持个性化定制，以便于快速评估和优化模型。

1.4.3 获取公开数据集

Ragas 支持从 hugging face hub 等平台下载公开数据集并自动进行格式对齐，确保评估框架和模型的高效使用。

1.5 集成 LLM 框架

轻松与多种 LLM 框架无缝集成，通过统一的接口对主流的 LLM 进行调用。同时，ragas 支持在不同框架之间轻松切换，还支持使用自定义框架。

1.6 缓存机制

通过缓存优化计算过程，减少重复计算和 API 调用，提升系统性能与响应速度。

1.6.1 静态缓存

基于 `lru_cache` 进行结果存储，对于相同输入返回缓存结果，减少计算资源消耗。

1.6.2 磁盘缓存

使用 `diskcache` 进行持久化存储，在程序重启后仍可读取缓存数据，适用于大规模数据处理场景。

1.6.3 自定义缓存策略

支持自定义缓存键生成方式，可根据具体业务需求调整缓存逻辑，确保灵活性和高效性。

1.7 自动重试

在任务执行失败时，提供自动重试机制，提升程序的稳定性，防止因临时性错误导致任务失败。

1.7.1 同步任务重试

针对同步任务，提供指数回退（exponential backoff）策略，确保在网络抖动或临时异常时能自动恢复。

1.7.2 异步任务重试

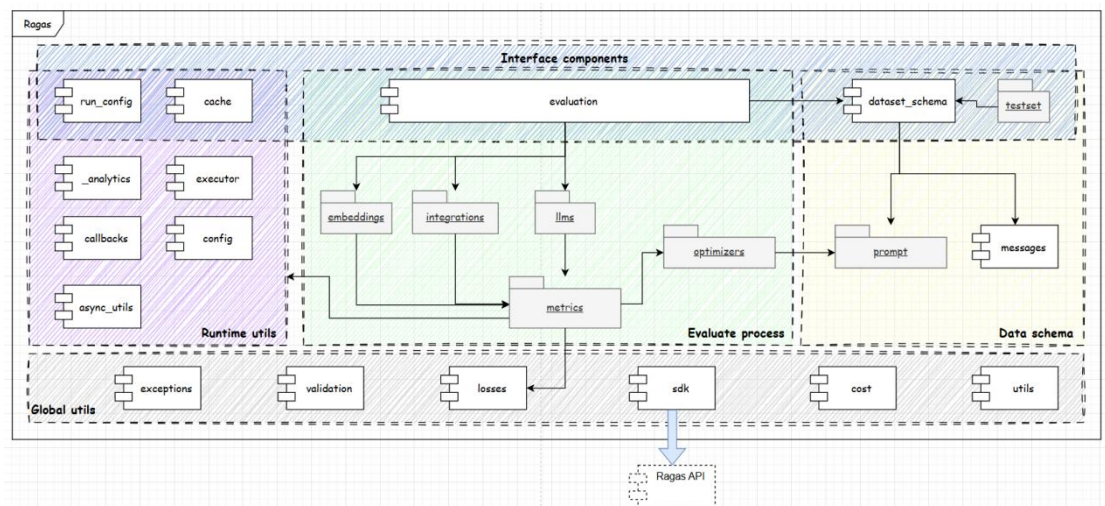
支持 `async` 方法的自动重试，适用于高并发任务，确保异步操作的可靠性和稳定性。

1.7.3 自定义重试规则

允许用户设置最大重试次数、等待时间等参数，灵活适配不同的任务需求，提升系统的可控性。

2. 架构图

源文件见 `images/system_structure.drawio`



3. 类的描述

包	模块	类	描述
ragas	_analytics	BaseEvent	所有事件的基类，包含用户 ID、Ragas 版本等通用信息。
		EvaluationEvent	表示一个评估事件，包含评估使用的指标、语言、评估

			类型等信息。
		TestsetGenerationEvent	表示一个测试集生成事件，包含测试集进化名称、百分比等信息。
		AnalyticsBatcher	用于批量收集和定期发送分析事件的类，能够在后台线程中定期刷新并发送事件。
		IsCompleteEvent	表示一个任务是否完成的事件，用于追踪函数是否成功执行完毕。
cache		CacheInterface	一个抽象基类，定义了缓存操作的标准接口
		DiskCacheBackend	实现了 <code>CacheInterface</code> ，它使用 <code>diskcache</code> 库来实现将缓存存储在磁盘上的功能
callbacks		ChainType	枚举类型，表示链的类型，如评估（EVALUATION）、度量（METRIC）、行（ROW）、Ragas 提示（RAGAS_PROMPT）。
		ChainRun	用于表示链执行的信息，包括运行 ID、父级 ID、输入输出数据、元数据等。
		ChainRunEncoder	用于将 ChainRun 实例编码为 JSON 格式的自定义编码器。
		RagasTracer	回调处理类，用于追踪链的执行过程，并记录每个运行的开始、结束及其相关信

			息。
		MetricTrace	用于表示度量追踪的类，存储度量的得分，并提供对这些得分的字符串表示。
		parse_run_traces	函数，用于解析链的运行过程，提取出所有行追踪数据并按顺序解析度量信息。
	config	DemonstrationConfig	一个 Pydantic 数据模型，用于配置与示范相关的设置
		InstructionConfig	一个 Pydantic 数据模型，用于配置与指令相关的设置，特别是与大语言模型（LLM）、损失函数和优化器相关的配置
	cost	TokenUsage	用于表示模型调用的 Token 使用情况，包括输入和输出的 token 数量。支持相加、成本计算等操作，并可以根据模型名称进行比较。
		get_token_usage_for_openai	获取 OpenAI 接口的 token 使用信息，通过提取 llm_result 中的 token_usage 字段中的数据来计算输入和输出的 token 数。
		get_token_usage_for_anthropic	获取 Anthropic 接口的 token 使用信息，适用于 ChatGeneration 类型的数据。根据 response_metadata 中的 token 信息计算输入和输出的 token 数。

	get_token_usage_for_bedrock	获取 Bedrock 接口的 token 使用信息，适用于 ChatGeneration 类型的数据。提取 response_metadata 中的 token 信息，计算输入和输出的 token 数。
	CostCallbackHandler	用于计算成本的回调处理器。记录 token 使用数据并计算模型运行的成本。支持单模型和多模型的成本计算，并能处理 token 使用情况的汇总。支持通过 per_model_costs 表格指定不同模型的 token 费用。
dataset_schema	BaseSample	评估样本抽象基类，继承了 pydantic 的 BaseModel，提供评估样本的数据校验和序列化功能
	SingleTurnSample	单轮交互的评估样本
	MultiTurnSample	多轮交互的评估样本。还包含了中间工具的交互
	RagasDataset	评估数据集的抽象基类，泛型类。提供与其他数据结构（比如列表、DataFrame 等）转换的接口
	EvaluationDataset	评估样本的数据集。基本就是泛型类指定了 *TurnSample 的数据集
	EvaluationResult	用于存储和处理评估结果。包含评估分数、原始数据

			集、二进制列、成本回调处理器等属性
		PromptAnnotation	提示词数据类，包含输入提示、输出提示和编辑后的输出提示
		SampleAnnotation	样本数据类，包含度量、提示词等属性
		MetricAnnotation	度量数据类。提供了从 JSON 文件或 API 加载注释、处理数据集等方法。
		SingleMetricAnnotation	单个度量数据类。
	evaluation		定义了评估功能的接口 <code>evaluate</code>，没有具体的类。 在给定的数据集上，根据模型、集成、包装、追踪等配置，对给定的指标进行评估，返回各项指标的结果。
	exceptions	RagasException	基础异常类，用于 Ragas 中的所有异常处理。
		ExceptionInRunner	当执行器中的线程抛出异常时引发的异常，通常用于表示在运行任务时出现了错误。
		RagasOutputParserException	当输出解析器无法解析输出时引发的异常。
		LLMDidNotFinishException	当 LLM 生成未完成时引发的异常，通常是由于生成未达到 <code>max_tokens</code> 限制导致。

		UploadException	当上传结果到应用程序失败时引发的异常，包含状态码和错误消息。
	executor	Executor	用于异步执行任务的类，带有进度条跟踪和错误处理功能。
	losses	Loss	抽象基类，定义损失计算的接口
		MSELoss	计算均方误差 (MSE)，用于回归任务
		BinaryMetricLoss	计算二分类损失 (accuracy 或 F1-score)
	messages	Message	表示一个通用的消息，包含消息内容 (content) 和可选的附加元数据 (metadata)。
		ToolCall	表示一次工具调用，包含工具名称 (name) 和一个字典类型的参数 (args)，参数名称和值可以是字符串、整数或浮动数。
		HumanMessage	表示来自人类用户的消息，继承自 Message，type 总是 "human"，并提供 pretty_repr 方法返回格式化的字符串表示。
		ToolMessage	表示来自工具的消息，继承自 Message，type 总是 "tool"，并提供 pretty_repr 方法返回格式化的字符串表示。

		AIMessage	表示来自人工智能的消息，继承自 Message ， type 总是 "ai" ，并包含一个可选的 tool_calls 列表（记录人工智能使用的工具调用）。支持 to_dict 和 pretty_repr 方法。
	run_config	RunConfig	一个数据类，用于配置执行操作时的超时、重试次数、等待时间等参数
	callbacks	new_group	回调链组，并提供 langchain 中管理回调链的管理器 CallbackManagerForChainGroup ， CallbackManagerForChainRun ，
		ChainType	回调链类型枚举。包括 EVALUATION 评估、 METRIC 指标生成、 ROW 数据行、 RAGAS_PROMPT 提示词
		ChainRun	数据模型类。描述了回调运行链的结构，包括运行 ID 、父运行 ID 、名称、输入、元数据、输出和子运行列表。
		ChainRunEncoder	继承 JSONEncoder ，用于将 ChainRun 对象和其他自定义类型（如 uuid.UUID 和 ChainType ）编码为 JSON
		RagasTracer	回调处理类，提供了对回调链运行情况的跟踪方法，并记录回调链开始结束的信息。

		MetricTrace	数据模型类。存储和表示度量分数
	validation		提供样本数据集的验证和清洗功能。验证包括验证数据集是否包含必须的类，数据集的项是否为 <code>dataset_schema</code> 里的样本的格式；清洗包括对列名的重命名，处理过时的列名。主要用于确保数据集在进行评估时符合预期的格式和要求，从而保证评估结果的准确性和可靠性。
	executor	Executor	异步工作的包装类。提供异步工作的启动、跟踪、错误处理以及简单的调度。
	sdk		Ragas API 服务支持
	utils		全局工具模块
ragas.embeddings	base	BaseRagasEmbeddings	词嵌入与配置的抽象基类，提供了嵌入文本和管理运行配置的方法，为具体的嵌入实现提供一个统一的接口和基础功能。
		LangchainEmbeddingsWrapper	Langchain 的词嵌入类特化
		HuggingfaceEmbeddings	Hugging face 的词嵌入类特化
		LlamaIndexEmbeddingsWrapper	Llama-index 的词嵌入类特化

	haystack_wrapper	HaystackEmbeddingsWrapper	Haystack 的词嵌入类特化
ragas.integrations	helicone	CacheConfig	Helicone 工具集成的缓存信息，包括信息与配置的有效时间
		HeliconeSingleton	Helicon 工具的配置信息，以单例模式存在，向全局暴露 helicone_config 变量
ragas.llms	base	BaseRagasLLM	Ragas 调用 LLM 接口的抽象基类。包含了 LLM 的通用运行配置、缓存方法以及运行跟踪与调用方法。能够方便扩展不同的 LLM 的交互。
		LangchainLLMWrapper	Langchain LLM 接口特化
		LlamaIndexLLMWrapper	Llama LLM 接口特化
	haystack_wrapper	HaystackLLMWrapper	Haystack LLM 接口特化
ragas.metrics	base	MetricType	枚举类。有 SINGLE_TURN 单轮对话指标和 MULTI_TURN 多轮对话指标
		MetricOutputType	枚举类。评估量化结果的类型。有 BINARY 二元，DISCRETE 离散型，CONTINUOUS 连续型和 RANKING 排名。
		Metric	评估指标的抽象基类。包含度量的名称和所需的列，并定义了初始化和评分方法。

		MetricWithLLM	使用语言模型进行评估的度量类。继承自 Metric 和 PromptMixin，包含语言模型和输出类型。
	_answer_correctness	QuestionAnswerGroundTruth	数据模型类。包含问题、LLM 回答以及真实值
		StatementsWithReason	数据模型类。包含陈述（statement）以及原因。
		ClassificationWithReason	数据模型类。包含陈述的 TP、FP 和 FN（即 True Positive 一类的缩写）的原因。
		CorrectnessClassifier	用于根据真实值和回答对陈述进行分类，并提供分类原因
		AnswerCorrectness	度量类，继承自 MetricWithLLM、MetricWithEmbeddings 和 SingleTurnMetric。用于衡量答案的正确性，结合事实性和语义相似性。也是这个模块向外暴露的接口。
	_answer_relevance	ResponseRelevanceOutput	LLM 回复的输入输出。
		ResponseRelevanceInput	
		ResponseRelevancePrompt	Prompt 数据结构，用于生成问题并识别回答是否含糊不清
		ResponseRelevancy	这是 MetricWithLLM、

			MetricWithEmbeddings 和 SingleTurnMetric 的组合类，用于评估回答的相关性。回答中不完整、冗余或不必要的信息会被扣分。分数范围从 0 到 1，1 为最佳。
		AnswerRelevancy	没有额外的属性或方法，只是继承了 ResponseRelevancy 的所有功能。对计算得分的方法进行了异步包装。
	_answer_similarity	SemanticSimilarity	计算生成的回答与真实值之间的语义相似度。使用 cross encoder score 来量化语义相似度。
		AnswerSimilarity	没有额外的属性或方法，只是继承了 ResponseRelevancy 的所有功能。对计算得分的方法进行了异步包装。
ragas.optimizers			
ragas.prompt	base	BasePrompt	抽象基类，定义了所有提示模板的通用接口，要求实现 generate 和 generate_multiple 方法
		StringIO	字符串输入数据结构，用于封装 LLM 输入的文本，并支持哈希计算。
		BoolIO	布尔输入数据结构，用于封装布尔值输入，并支持哈希

			计算。
		StringPrompt	评估数据集的抽象基类，泛型类。提供与其他数据结构（比如列表、 DataFrame 等）转换的接口
	few_shot_pydantic_prompt	ExampleStore	抽象基类，定义了示例存储的接口，包括 get_examples 和 add_example 方法。
		InMemoryExampleStore	内存示例存储实现，使用嵌入向量存储示例，并基于余弦相似度进行检索。
		FewShotPydanticPrompt	Few-shot 任务提示类，继承 PydanticPrompt ，并结合 ExampleStore 提供示例支持，提高 LLM 生成质量。
	mixin	PromptMixin	Mixin 类，用于管理类中的 PydanticPrompt 对象，支持获取、设置、适配、保存和加载 Prompt 。
	multi_modal_prompt	ImageTextPrompt	继承自 PydanticPrompt ，用于生成带有图像和文本的提示，支持生成多个输出，并处理输入输出的解析。它将文本输入转换为多种格式并适配 LLM。
		ImageTextPromptValue	用于表示和处理带有图像和文本的提示值，支持将图像和文本数据转换为消息格式，同时能处理图像的 URL 或 Base64 编码图像。它也提供了文本和图像的串联和

	pydantic_prompt		验证方法。
		PydanticPrompt	继承自 BasePrompt，提供生成与处理模型输入输出的功能，并支持 Pydantic 数据模型。
		FixOutputFormat	继承自 PydanticPrompt，用于修复不符合提示约束的输出字符串并返回正确格式的输
		RagasOutputParser	用于解析 Ragas 输出，尝试修复解析错误，并使用 FixOutputFormat 进行重试。
		ToTranslate	用于表示翻译任务的输入数据模型，包括目标语言和待翻译的语句列表。
		Translated	用于表示翻译任务的输出数据模型，包括翻译后的语句列表。
		TranslateStatements	继承自 PydanticPrompt，用于执行翻译任务，将输入的语句翻译成目标语言，并确保输出的行数与输入的行数一致。
ragas.testset.synthesizers	base	QueryLength	枚举类型，表示查询的长度，支持三种长度：LONG, MEDIUM, SHORT。
		QueryStyle	枚举类型，表示查询的风格，支持四种风格：MISSPELLED, PERFECT_GRAMMAR,

			POOR_GRAMMAR, WEB_SEARCH_LIKE。
		BaseScenario	基类，用于表示生成测试样本的场景。包含节点列表、查询风格、查询长度和与场景相关的人物角色。
		BaseSynthesizer	基类，用于合成场景和样本。通过指定知识图谱、人物角色等生成场景，并生成相应的样本。继承自 PromptMixin，支持提示管理。
		Scenario	泛型类型，约束为 BaseScenario 类的子类，用于指定场景类型。
	generate	TestsetGenerator	用于生成评估数据集的类，基于给定的场景、文档、查询分布等参数生成测试样本。支持从 Langchain 和 LlamaIndex 加载 LLM 和嵌入模型。
	prompts	ThemesPersonasInput	用于表示主题和人物的输入数据模型，包括主题列表和人物列表。
		PersonaThemesMapping	用于表示主题和人物的匹配关系数据模型，包含人物名称与相关主题的映射关系。
		ThemesPersonasMatchingPrompt	继承自 PydanticPrompt，用于根据人物角色描述将人物与相关主题进行匹配。
	testset_sc	TestsetSample	表示测试集中的一个样本，

	hema		包含评估样本（单轮或多轮）和生成该样本的合成器名称。
		TestsetPacket	用于将测试集样本打包上传至服务器的模型，包含原始样本列表、运行 ID 和创建时间。
		Testset	表示包含多个测试样本的测试集，支持转换为评估数据集、计算总使用的 token 数量和成本，并可上传至服务器。
ragas.testset.synthesizers.multi_hop	abstract	MultiHopAbstractQuerySynthesizer	用于从给定的知识图谱合成抽象的多跳查询的合成器类，涉及主题、人物角色匹配、概念组合等任务。
	base	MultiHopScenario	表示多跳查询场景，包含节点、查询组合、查询风格、查询长度以及与场景相关的人物角色信息。
		MultiHopQuerySynthesizer	用于从给定知识图谱生成多跳查询的合成器类，负责生成查询场景、匹配人物与主题、创建查询组合等。
	prompts	ConceptsList	定义包含多个概念列表的数据模型，用于指定每个节点的概念列表及要生成的最大概念组合数。
		ConceptCombinations	定义概念组合的数据模型，包含一组概念的组合。
		ConceptCombination	继承自 PydanticPrompt，用

		Prompt	于根据给定的概念列表生成概念组合的提示，支持从不同节点的概念中形成组合。
		QueryConditions	定义查询条件的数据模型，包括人物角色、主题、查询风格、查询长度和上下文。
		GeneratedQueryAnswer	定义生成的查询和答案的数据模型。
		QueryAnswerGenerationPrompt	继承自 PydanticPrompt，用于根据给定条件（如人物角色、主题、查询风格等）生成多跳查询及其答案的提示。
	specific	MultiHopSpecificQuerySynthesizer	用于生成基于特定关键短语的查询，通过选择知识图谱中重叠的实体并生成查询。
ragas.testset.synthesizers.single_hop	base	SingleHopScenario	用于表示单跳查询场景，包含查询主题、节点、风格、长度和人物角色等信息。
		SingleHopQuerySynthesizer	用于生成单跳查询的合成器类，负责生成查询场景、选择查询条件并生成查询。
	prompts	QueryCondition	定义查询条件的数据模型，包括人物角色、查询主题、查询风格、查询长度和上下文。
		GeneratedQueryAnswer	定义生成的查询和答案的数据模型。
		QueryAnswerGenerationPrompt	继承自 PydanticPrompt，用于根据给定的查询条件（如

			人物角色、主题、风格、长度等）生成单跳查询和答案的提示。
	specific	SingleHopScenario	表示单跳查询场景，包含查询的主题（term）以及相关的节点、人物角色、查询风格、长度等信息。
		SingleHopSpecificQuerySynthesizer	用于生成特定关键字查询的合成器类，通过选择包含特定实体的节点，生成相关查询，并匹配人物角色与实体。
ragas.testset.transforms.extractors	embeddings	EmbeddingExtractor	负责从知识图谱中的节点提取嵌入 (embedding)，继承自 Extractor。包含 property_name（存储嵌入的属性）、embed_property_name（需要嵌入的文本属性）和 embedding_model（用于生成嵌入的模型）。
		SummaryExtractor	提取文本摘要，使用 SummaryExtractorPrompt 进行摘要生成，并限制摘要不超过 10 句话。
	llm_based	KeyphrasesExtractor	提取文本中的关键短语，使用 KeyphrasesExtractorPrompt 生成最多 5 个关键短语。
		TitleExtractor	从文本中提取标题，使用 TitleExtractorPrompt 进行提取。

		HeadlinesExtractor	提取文本中的重要标题，使用 HeadlinesExtractorPrompt 进行提取，侧重于 2 级和 3 级标题，并限制最大提取数量。
		NERExtractor	进行命名实体识别（NER），使用 NERPrompt 提取最多 10 个命名实体，如人名、地名、组织名等。
		TopicDescriptionExtractor	提取文本的主题描述，使用 TopicDescriptionPrompt 生成简明的主题概述。
		ThemesExtractor	提取文本的主要主题和概念，使用 ThemesAndConceptsExtractorPrompt 提取最多 10 个关键主题和概念，如 "人工智能"、"自动化"、"数据分析" 等。
	regex_based	RegexBasedExtractor	通过正则表达式从文本中提取特定内容，适用于匹配 URL、邮箱、Markdown 标题等。
		links_extractor	提取文本中的 URL（包括 http://、https:// 和 www. 开头的链接）。
		emails_extractor	提取文本中的电子邮件地址。
		markdown_headings_extractor	提取 Markdown 文档中的标题（以 # 开头的行，支持 1-

			6 级标题) 。
ragas.testset. transforms.relationship_builders	cosine.py	CosineSimilarityBuilder	通过余弦相似度计算节点间的相似关系，建立相似度高于阈值的关系。
		SummaryCosineSimilarityBuilder	专门用于基于摘要嵌入计算文档节点的余弦相似度，并筛选具有摘要嵌入的节点进行关系构建。
	traditional	JaccardSimilarityBuilder	计算知识图谱中节点属性的 Jaccard 相似度，并建立相似度高于阈值的关系。
		OverlapScoreBuilder	计算节点属性之间的重叠得分，使用字符串距离度量（如 Jaro-Winkler），并筛除噪声数据以构建关系。
ragas.testset. transforms splitters	headline	HeadlineSplitter	基于文档中的标题拆分文本，将其划分为合理长度的片段，同时维护层级和顺序关系。
ragas.testset. transforms	base	BaseGraphTransformation	抽象基类，用于对 KnowledgeGraph 进行转换。
		Extractor	提取器基类，用于从 KnowledgeGraph 的节点中提取特定属性。
		LLMBasedExtractor	结合 LLM 的 Extractor，用于处理和生成文本数据。
		Splitter	拆分器基类，将 KnowledgeGraph 的节点拆

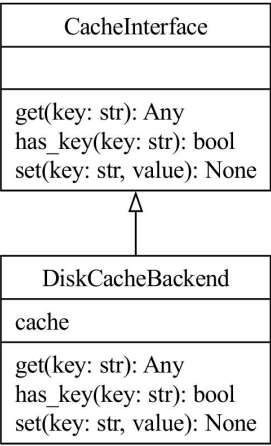
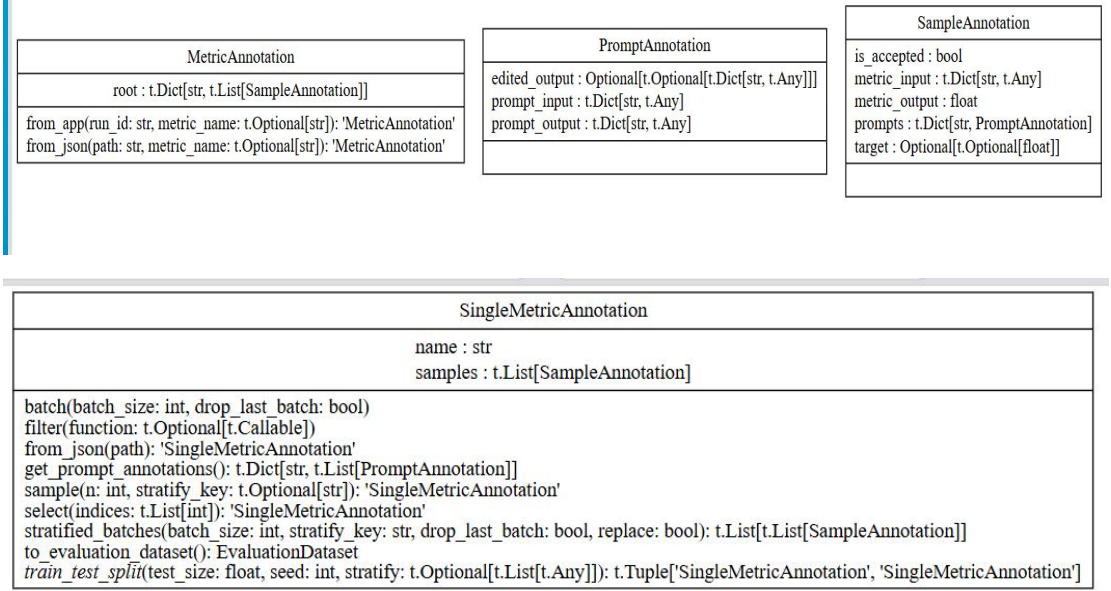
			分成更小的单元。
		RelationshipBuilder	关系构建器基类，用于在 KnowledgeGraph 中构建新的关系。
		NodeFilter	过滤器基类，用于根据自定义条件筛选 KnowledgeGraph 的节点。
		LLMBasedNodeFilter	结合 LLM 的 NodeFilter，用于基于自然语言理解的节点过滤。
	engine	Parallel	并行应用多个图转换的集合类。
		Transforms	可用于 KnowledgeGraph 变换的类型定义，包括 BaseGraphTransformation、Parallel 或列表形式的 BaseGraphTransformation。
	filters	QuestionPotentialInput	定义了用于评估节点内容相关性和问题生成潜力的输入数据，包括文档摘要、节点内容和评分标准。
		QuestionPotentialOutput	定义评分输出，范围为 1 到 5。
		QuestionPotentialPrompt	评分提示，基于文档摘要和节点内容计算相关性评分。
		CustomNodeFilter	过滤低相关性节点的自定义筛选器，基于评分标准 (rubrics)，如果评分低于 min_score，则移除该节点。

ragas.testset	graph	UUIDEncoder	自定义 JSON 编码器，确保 uuid.UUID 类型的对象可以被序列化为字符串。
		NodeType	知识图谱中的节点类型枚举，包括 UNKNOWN、DOCUMENT 和 CHUNK。
		Node	知识图谱的基本单元，包含唯一标识 (id)、属性 (properties) 和类型 (type)。
		Relationship	代表知识图谱中两个节点之间的关系，支持单向和双向关系，并附带属性。
		KnowledgeGraph	知识图谱的核心类，管理节点 (nodes) 和关系 (relationships)，提供添加、保存、加载、查找和删除节点的功能。
	persona	Persona	代表一个角色，包括 name (名称) 和 role_description (角色描述)。
		PersonaGenerationPrompt	通过提供的摘要，生成与内容相关的一个独特 Persona (角色)，包含名称和简要描述。
		PersonaList	由多个 Persona 组成的列表，支持通过名称索引特定 Persona。

4. 包的类图

images/dotfiles 下所有包和模块的类图 graphviz 源文件。但目前还未美化（可以让 llm 来做）

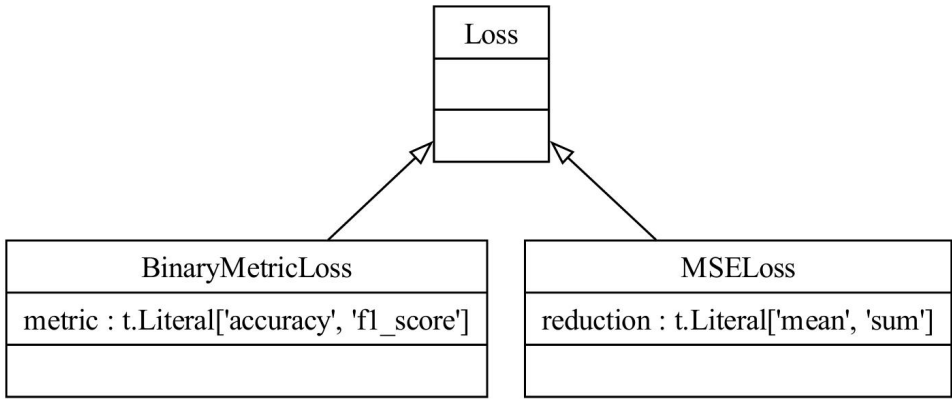
4.1 Dataset_schema



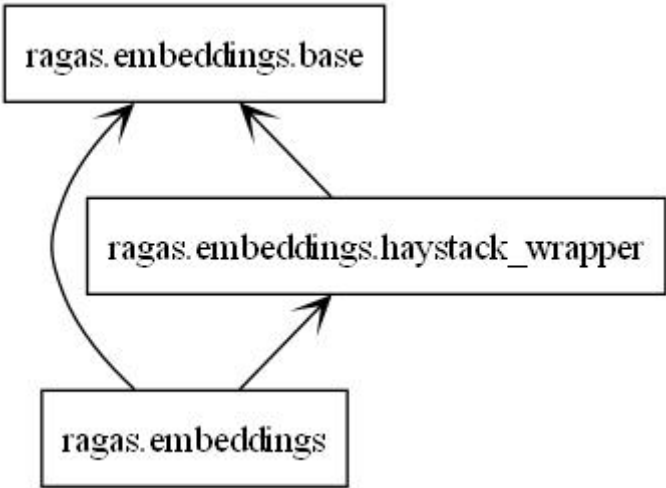
RunConfig
exception_types : t.Union[t.Type[BaseException], t.Tuple[t.Type[BaseException], ...]] log_tenacity : bool max_retries : int max_wait : int max_workers : int rng seed : int timeout : int

DemonstrationConfig
embedding enabled : bool technique : t.Literal['random', 'similarity'] threshold : float top_k : int
validate_embedding(v)

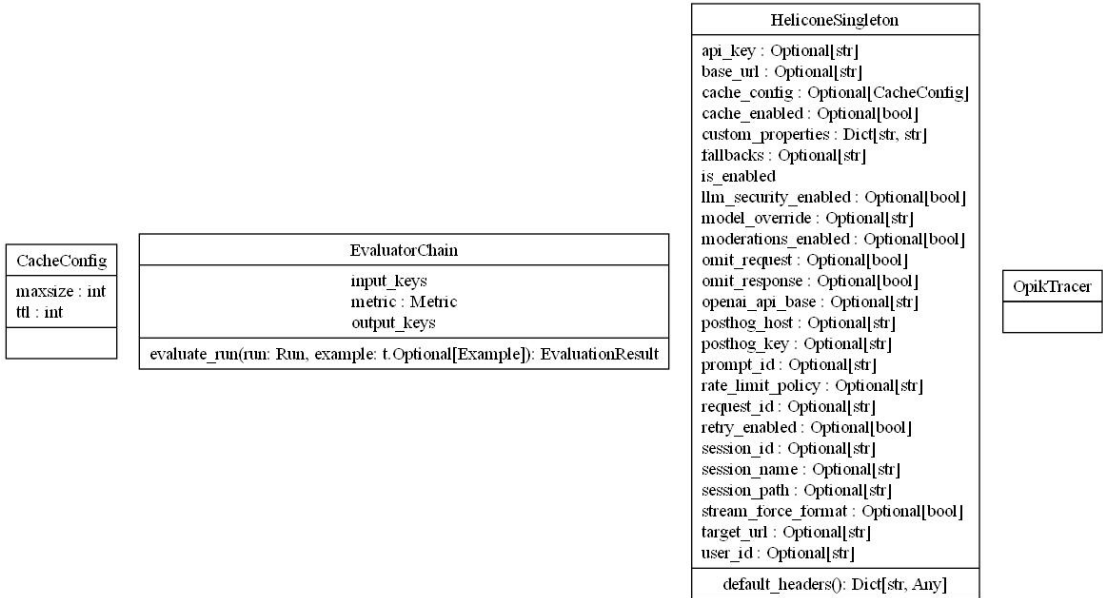
InstructionConfig
enabled : bool llm : BaseRagasLLM loss : Optional[t.Optional[Loss]] optimizer : Optimizer optimizer_config : t.Dict[str, t.Any]



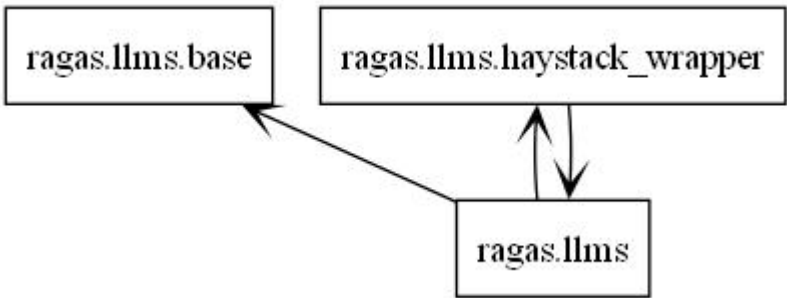
- embeddings 包
 - 包图:



- integrations 包：
 - 类图：

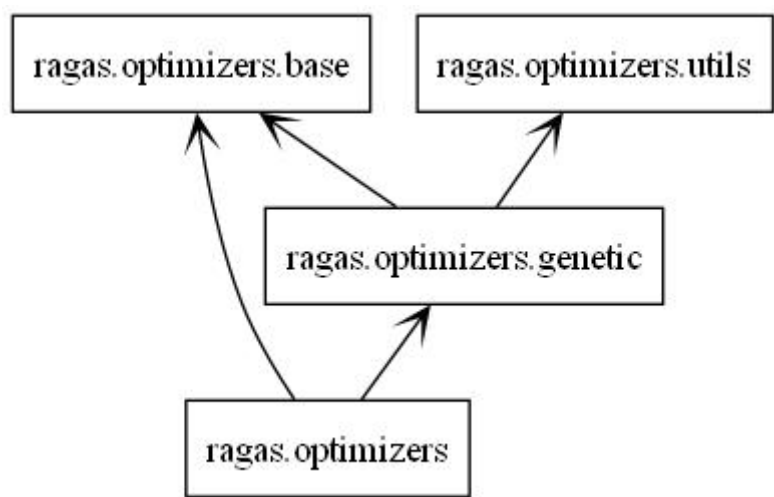


- llms 包：
 - 包图：



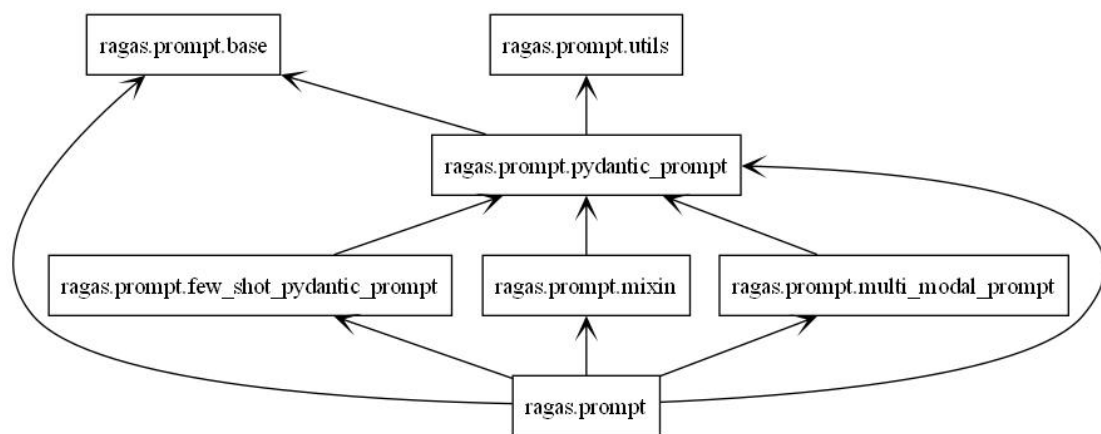
- optimizers 包：

- 包图:



- prompt 包:

- 包图:



5. 功能和类的对应关系表

功能名称	实现模块	实现方法
用于评估样本，提供数据验证和序列化功能。	BaseSample SingleTurnSample MultiTurnSample	to_dict() get_features()
用于评估数据集，提供与其他数据结构转换的	RagasDataset EvaluationDataset	validate_samples(samples) get_sample_type() to_hf_dataset()

接口		to_pandas()
用于存储和处理评估结果	EvaluationResult	to_pandas() total_tokens() total_cost() upload()
输入提示、输出提示和编辑后的输出提示，主要用于记录和提示处理相关信息。	PromptAnnotation	get_item()
用于样本数据度量，同时用于注释样本的信息	SampleAnnotation MetricAnnotation SingleMetricAnnotation	to_evaluation_dataset()
数据及验证与准备	ragas.validation	remap_column_names() validate_required_columns() validate_supported_metrics()
评估指标初始化	Metric, MetricWithLLM,MetricWithEmbeddings	init() llm() embeddings()
执行评估	Executor	submits() results()
单轮评估	SingleTurnMetric	single_turn_ascore()
多轮评估	MultiTurnMetric	multi_turn_ascore()
结果收集与	EvaluationResult	total_tokens()

处理		total_cost()
跟踪运行信息	BaseCallbackHandler, RagasTracer	on_chain_start() on_chain_end() to_jsons()
评估模型输出质量	MetricWithLLM, MetricWithEmbeddings	init() _optimize_instruction() _optimize_demonstration() train()
设置默认指标	ragas.metrics	answer_relevancy() context_precision() faithfulness() context_recall()
数据集类型转换	EvaluationDataset	from_list()
计算成本	BaseCallbackHandler, CostCallbackHandler	on_llm_end() total_cost() total_tokens()
缓存	ragas.cache	_make_hashable() _generate_cache_key() sync_wrapper() async_wrapper()
自动重试	ragas.run_config	add_retry() add_async_retry()
损失函数	ragas.losses	__call__() __get_pydantic_core_schema__()

6. 代码注释

包	模块	标注的代码 行数	标注 人
ragas	evaluation	9	@刘存玺
	executor	4	@刘存玺
	callbacks	1	@刘存玺
ragas.integrations	helicone	1	@刘存玺
ragas.llms	base	27	@刘存玺
ragas.metrics	_answer_correctness	12	@刘存玺
	_answer_relevance	5	@刘存玺
ragas.optimizers	base	17	@余明晖
	genetic	11	@余明晖
ragas.prompt	base	6	@余明晖
	mixin	12	@余明晖
ragas	config	2	@Pchan

			潘威
	cache	1	@Pchan
	run_config	7	@Pchan
	losses	5	@Pchan
rages.Dataset_schema	Sample	12	@王荣铮
	Dataset	4	@王荣铮
	Annotation	6	@王荣铮

7. 阅读收获

7.1 类的描述

使用 IDE 的 Copilot 插件可以快速生成当前模块中的类的描述信息。而且 chat 中的类引用是深度集成到 IDE 的，可以直接点击跳转。哪里不会点哪里。

相较于直接将代码复制粘贴到 LLM，copilot 对项目结构的把握更加精准，生成的描述也更贴近源码阅读任务的期望。最重要的是代码文本不会占用特别多的上下文长度，可以认为基本没有上下文长度的限制，可以随意进行多轮对话。



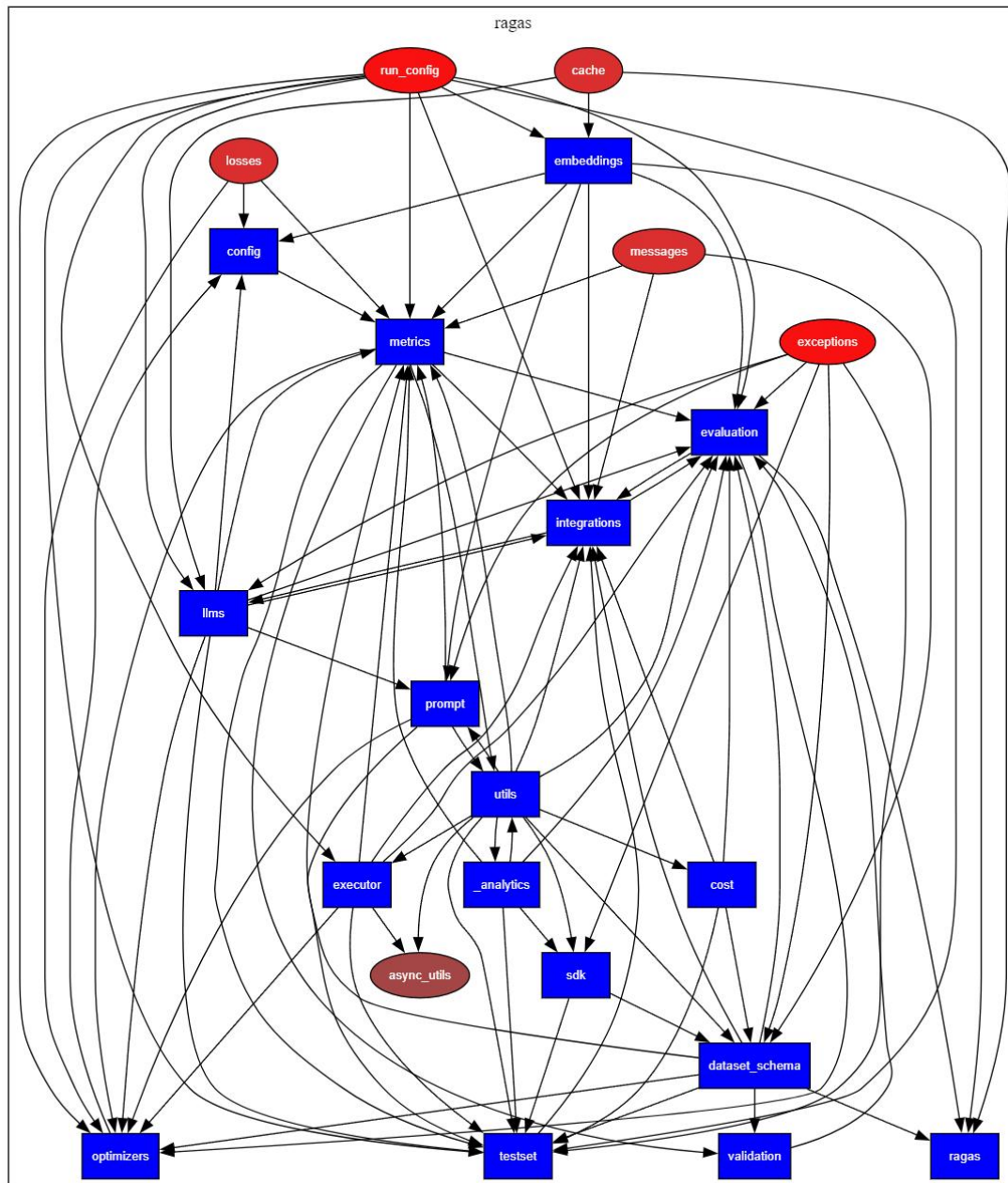
7.2 UML 图生成

使用 `pyreverse` 可以生成模块依赖图和类图，默认是 `dotfile` 格式，可以用 `graphviz` 渲染。缺点是 `pyreverse` 生成的图像粒度太细，会具体到最末端的模块。

而使用 `pydeps` 可以控制依赖深度，从而控制依赖图的复杂度。不过缺点是生成的图并非标准的 UML 图，但是对于把握项目架构还是有很大帮助的。

生成模块依赖图的命令如下：

```
python
pydeps ../src/ragas\ --max-bacon=2 --cluster --max-cluster-size=3
--keep-target-cluster --rmprefix ragas. --max-module-depth=2
```

7.3 包组织混乱

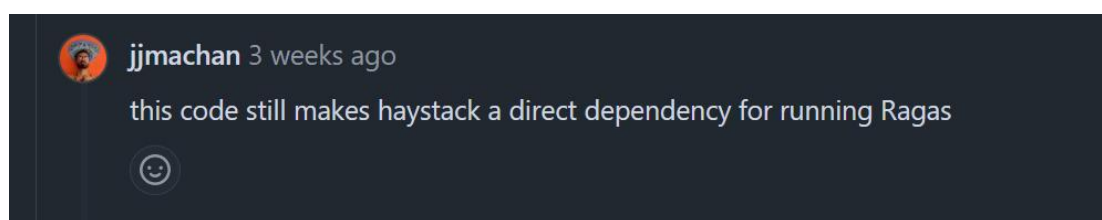
Embeddings 包与 LLM 包里的模块组织比较混乱。二者混乱的原因类似，以 embeddings 包为例，主要体现在以下两点：

1. 派生类位置不统一

Embedding 包中的派生类，主要组织在 `base.py` 模块下。而最近（2025/2/20）的一次 commit 中更新了 `haystack_wrapper` 的派生类，实现在 `base.py` 之外的 `haystack_wrapper.py` 中。而 `haystack_wrapper` 的实现和 `base.py` 中一系列的派生类并无太大区别，地位也是一致的，想不出有什么这样做的必要。

查看了相关 PR 信息，可能是 ragas 维护者对 haystack 的依赖持谨慎态度，故新加一

个模块单独实现。



2. 真假工厂方法

Embeddings 包中的 `base` 模块提供了一个名为 `embedding_factory` 的接口，似乎是创建 `embedding` 类的工厂方法。但实际上这个“工厂”只会创建一个 `Langchain` 的 `embedding` 产品，作为外部 `embedding` 默认参数。可能是为后续扩展做准备。

7.4 optimizers 包

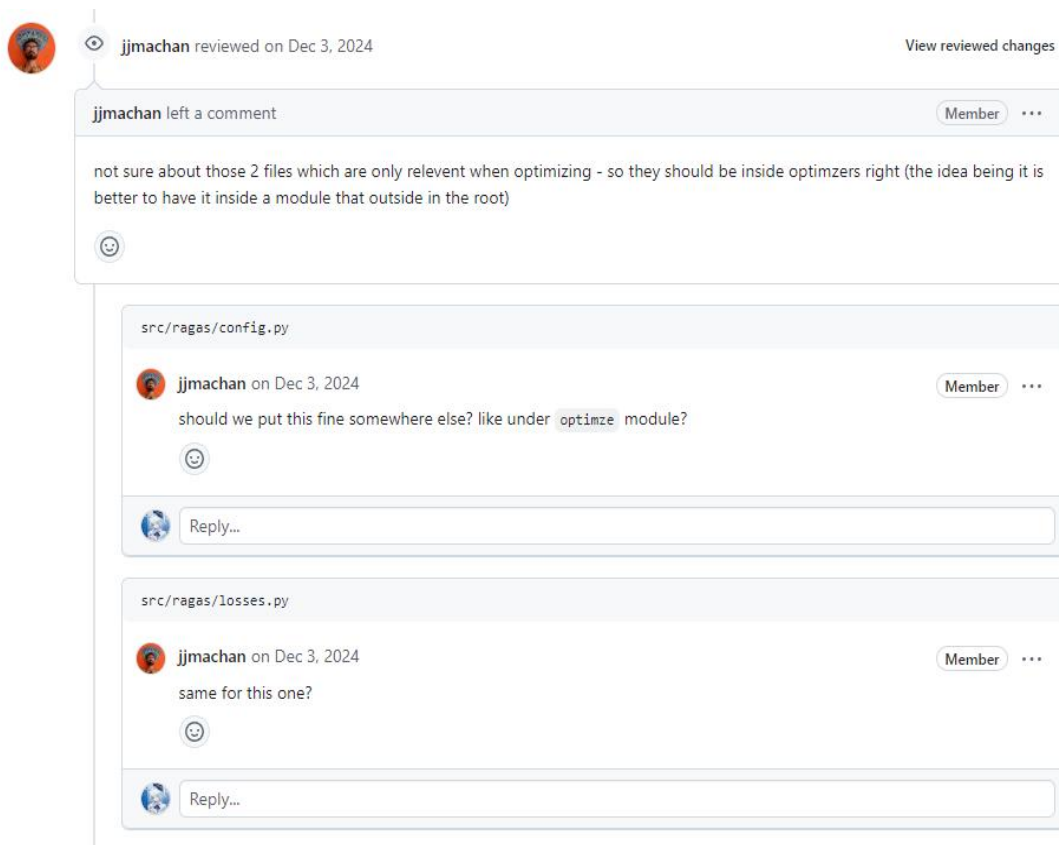
在这个代码包中，设计模式和设计思想的运用非常丰富，涵盖了多个层次和方面。首先是**抽象工厂模式**和**策略模式**。`Optimizer` 类作为一个抽象基类，定义了优化器的接口，具体的优化器（如 `GeneticOptimizer`）则实现了这个接口。这种设计使得不同的优化算法可以在不修改客户端代码的情况下进行替换，符合开闭原则。同时，`optimize` 方法可以被视为一个策略接口，具体的优化算法是实现该策略的具体类，运行时可以根据需要选择不同的优化策略。

其次，策略模式也有在 `GeneticOptimizer` 中体现。`optimize` 方法定义了一个算法的骨架，具体的步骤（如初始化种群、反馈变异、交叉变异、适应度评估）由子类实现。这种设计允许在不改变算法结构的情况下，重新定义算法的某些步骤，增强了代码的灵活性和可扩展性。

此外，**观察者模式**也被用于处理事件通知。代码中使用了 `Callbacks` 来跟踪优化过程中的进度和状态，这种设计使得在优化过程中可以动态添加或移除观察者，增强了代码的可扩展性。

在代码的组织和结构上，**依赖注入**和**封装**的思想贯穿始终。`GeneticOptimizer` 类中的 `llm` 和 `metric` 是通过构造函数或方法参数注入的，而不是在类内部直接创建。这种设计使得类更加灵活，易于测试和维护。同时，代码中大量使用了封装的思想，将数据和操作数据的方法封装在类中，使得外部只需调用接口即可，隐藏了内部实现细节。

有两个文件(`config.py` 和 `losses.py`)仅在优化时有使用，即它们只与优化时相关，但他们并没有被放在 `optimizers` 包下，关于这一点，作者是这样解释的：



7.5 单例模式和全局变量

项目中很多地方使用了全局变量的方法来实现单例模式。这一点和 Java 等强面向对象语言很不一样。使用时只需要 `import` 这个全局变量就行了。只是这样的话在 `import` 的时候就加载了 `instance`，并非 `lazy loading`。不过都用 `python` 了，追求的是快速实现，再慢一点也无所谓。

```
python
# Singleton class definition
class HeliconeSingleton:
    ...

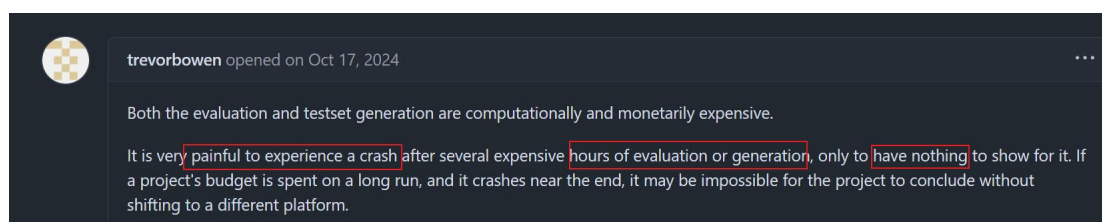
# Singleton instance, implemented as a module-level global
variable
helicone_config = HeliconeSingleton()
```

7.6 Diskcache

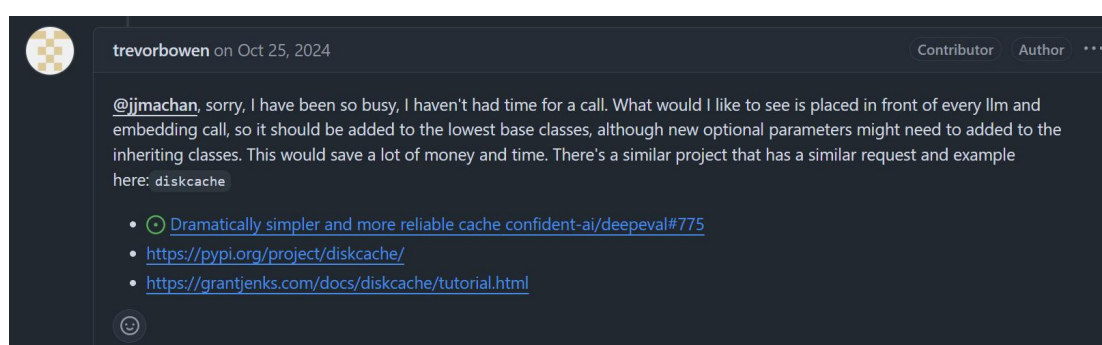
这部分代码主要涉及配置管理，并不直接参与 Ragas 的核心运行流程。比较值得注意的是，缓存功能的实现完全依赖于 `diskcache` 库，而没有采用自定义的缓存机制，因此整体逻辑较为直接，阅读难度不高。

DiskCache 是一个高效的磁盘和文件缓存库，可以提供比内存缓存更持久的存储方案，同时保持接近内存缓存的访问速度。在 ragas 中可以使用 DiskCacheBackend 对模型回复进行缓存。

还是有点怀疑 diskcache 的必要性，因为一般用于评估的语料数据集都不会很大（和隔壁 CV 相比），然后就在 issue 里看到有老哥破防了：

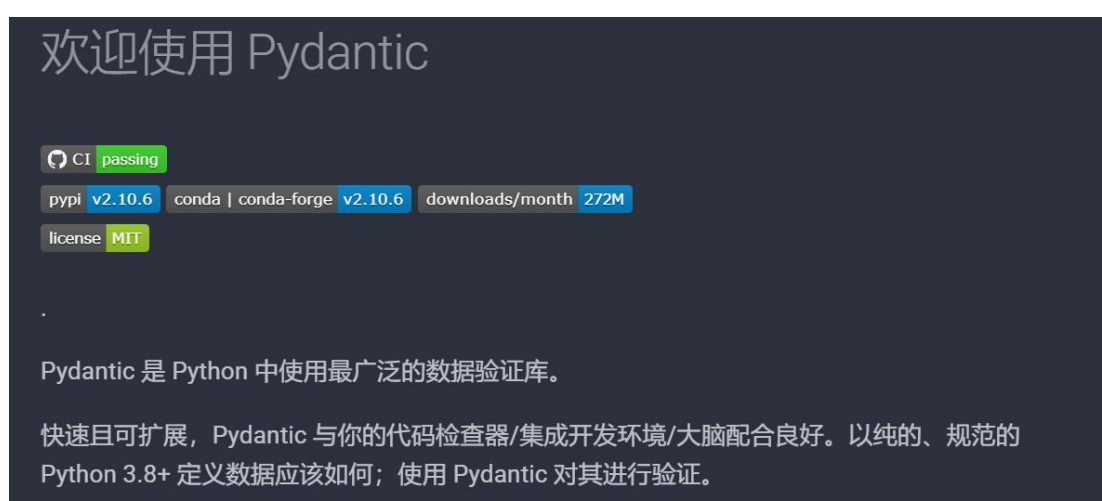


这位老哥也是见多识广，直接贴了 diskcache 的解决方案。Ragas 的维护者也是很快就将这个模块加入了进来。



7.7 Pydantic

此外，代码中大量使用了 Pydantic 进行数据验证和模型定义，我也是搜索了解了一下 Pydantic。



相较于 ragas 代码本身，这部分收获更多来源于它所使用的库。

不过，pydantic 也只是一个伪数据类型的限定工具，并不是 python 语言本身的特性。

使用这个库只能在一定程度上保证项目开发的类型规范性。

7.8 Dataset_schema

首先，值得我们学习的是代码结构清晰地分离了评估样本的不同类型，比如单独设计了用于单轮交互和多轮交互的评估样本类 `SingleTurnSample` 和 `MultiTurnSample`，这种设计提升了在后续评估中评估形式的灵活性和代码整体的可读性。其次，数据的输入输出方式非常灵活，`RagasDataset` 提供了多种格式的转换方法，包括将数据集导出为 JSONL、CSV 等格式，以及从这些格式中加载数据，可以适配各种形式的输入以及将评估结果以各种形式导出，提高了其泛用性。同时，在数据方面实现了异常处理机制的实现，例如在上传结果时对不同状态码的处理，能够有效捕获可能出现的错误并给予相应准确提示，增强了程序的鲁棒性和可维护性。整体来看，这段代码在样本处理、数据交换及错误管理等方面提供了很好的设计思路。